Systems Track

Interview Preparation Series • Professional Edition

Computer Networks

03

Compiled with LaTeX • June 17, 2026

harsha.somisetty01@gmail.com

In This Edition

> **Target:** Deep understanding from first principles + rapid recall under pressure. **Covers:** OSI/TCP-IP, TCP/UDP, HTTP/HTTPS, TLS/SSL, DNS, REST, gRPC, WebSockets, HTTP/2&3, Load Balancing, CORS, and more.

1 Overview --- What It Is

Computer networking is the discipline of **connecting computing devices so they can exchange data**. Every product at scale — from a URL you type to a microservice call 10 hops away — relies on a layered stack of protocols, each solving one narrow problem.

Key vocabulary before diving in:

Term	Definition
Protocol	Agreed rules for formatting and transmitting data
Packet	Fixed-size chunk of data with header metadata
Socket	(IP, Port) pair — the addressing unit for a connection
Latency	Time for one packet to travel source → destination
Bandwidth	Maximum data volume per unit time on a link
Throughput	Actual data successfully delivered per unit time
MTU	Maximum Transmission Unit — largest packet a link can carry (typically 1500 bytes on Ethernet)

2 Why It Exists

Problem: Two machines separated by any distance need to exchange bytes reliably, efficiently, and securely — even over unreliable physical media (copper, fiber, radio).

Solution through layering: Each layer solves exactly one problem and hands off a clean abstraction to the layer above. This is the core engineering insight behind all networking.

- Physical media is unreliable → **data link** adds error detection
- Links only carry point-to-point frames → **network** adds global addressing and routing
- Routing drops/reorders packets → **transport** adds reliability or ordering
- Reliability is generic; apps need different contracts → **application** protocols encode semantics

This separation of concerns is why you can swap WiFi for Ethernet without changing your HTTP client.

3 Why FAANG Cares (Per Company)

Company	Specific Concern
Google	Invented QUIC (now HTTP/3), SPDY (became HTTP/2), gRPC. Operates one of the largest private WANs (B4). Networking is core infrastructure.
Meta	Runs one of the largest CDNs + social graph. L4/L7 load balancing at petabyte scale. QUIC adoption early. Open-sourced Proxygen, Katran.

Company	Specific Concern
Amazon (AWS)	Sells networking primitives: VPC, ELB, Route 53, CloudFront, Direct Connect. Candidates must understand the product they are building on.
Netflix	30%+ of peak US internet traffic. Deep CDN (Open Connect), adaptive bitrate streaming, chaos engineering on network paths.
Apple	iMessage, FaceTime, iCloud — all require secure, efficient transport. Certificate pinning, TLS, push notifications (APNs).
Uber/Airbnb	Real-time dispatch, WebSocket connections, microservice mesh, service discovery.

FAANG universal: System design interviews are networking interviews in disguise. "Design a URL shortener" → DNS, HTTP redirects. "Design a chat app" → WebSockets, long-polling, scaling connections. "Design a video platform" → CDN, adaptive streaming, load balancing.

4 Core Concepts

4.1 1. OSI Model vs TCP/IP Model

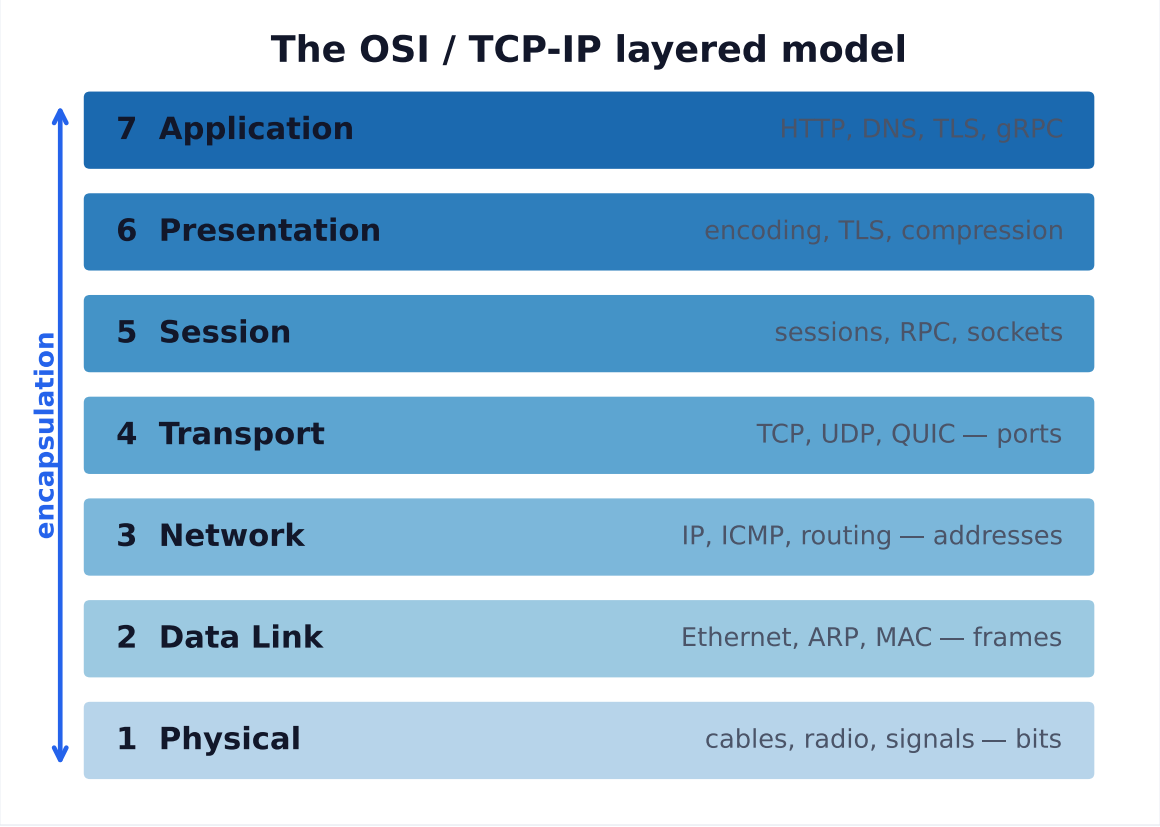


Figure 1. Each layer adds a header and treats the layer above as opaque payload. Interview framing: name the layer, its addressing unit, and a protocol.

OSI Model (7 Layers)

The **Open Systems Interconnection** model is a conceptual framework — not an implementation. It exists to give engineers a common vocabulary.

```
Layer 7 — Application    [HTTP, DNS, SMTP, FTP, gRPC]
Layer 6 — Presentation   [TLS/SSL, encoding, compression, encryption]
Layer 5 — Session        [session management, RPC, NetBIOS]
Layer 4 — Transport      [TCP, UDP — port numbers, reliability]
Layer 3 — Network        [IP, ICMP, routing]
Layer 2 — Data Link      [Ethernet, MAC addresses, switches, ARP]
Layer 1 — Physical       [bits on wire, fiber, radio, voltage levels]
```

TCP/IP Model (4 Layers)

The practical model actually implemented in the internet:

```python
Application     = OSI Layers 5+6+7
Transport       = OSI Layer 4
Internet        = OSI Layer 3
Network Access  = OSI Layers 1+2
```

Interview takeaway: OSI is for vocabulary and troubleshooting; TCP/IP is what runs on real systems. When asked "what layer does X operate at?" — use OSI layer numbers.

Data encapsulation flow:

```
App writes data
   ↓ [L7] HTTP adds method, headers, URL
   ↓ [L6] TLS encrypts the payload
   ↓ [L4] TCP adds source/dest port, seq number → SEGMENT
   ↓ [L3] IP adds source/dest IP → PACKET
   ↓ [L2] Ethernet adds MAC addresses → FRAME
   ↓ [L1] Transmits bits on wire
```

On the receiving side, each layer strips its header and passes the payload up.

4.2 2. IP Addressing, Subnetting, CIDR, NAT

IPv4 Addressing

- 32 bits = 4 octets = e.g. `192.168.1.100`
- Total address space: 2^32 = ~4.3 billion addresses (exhausted!)
- **Classes (historical, replaced by CIDR):**

```
Class A: 1.0.0.0   - 126.255.255.255  /8  (/8 → 16M hosts)
Class B: 128.0.0.0 - 191.255.255.255  /16 (→ 65K hosts)
Class C: 192.0.0.0 - 223.255.255.255  /24 (→ 254 hosts)
```

CIDR (Classless Inter-Domain Routing)

Notation: `IP/prefix_length` — the prefix length tells how many bits are the **network portion**.

```
192.168.1.0/24
  ↑             Network: 192.168.1   (24 bits)
              Host:    .0 to .255  (8 bits → 2^8 = 256 addresses, 254 usable)

10.0.0.0/8   → 2^24 = 16,777,216 addresses
10.0.0.0/16  → 2^16 = 65,536 addresses
10.0.0.0/28  → 2^4  = 16 addresses (14 usable, minus network + broadcast)
```

Subnet mask: `/24` = `255.255.255.0` (binary: 24 ones followed by 8 zeros)
Usable hosts = 2^(32 - prefix) - 2 (subtract network address and broadcast)

Private IP Ranges (RFC 1918)

```
10.0.0.0/8          (10.x.x.x)
172.16.0.0/12       (172.16.x.x - 172.31.x.x)
192.168.0.0/16      (192.168.x.x)
127.0.0.0/8         (loopback — localhost)
```

These are **not routable on the public internet** — this is why NAT exists.

NAT (Network Address Translation)

Problem: Private addresses can't be routed on the internet. Companies have thousands of devices sharing a handful of public IPs.

Solution: NAT router rewrites packet headers, maintaining a translation table.

```
Internal device            NAT Router            Internet Server
192.168.1.5:4500  ──►  203.0.113.1:12345  ──►  93.184.216.34:80
                        (rewrites src IP+port)
                  ◄──  203.0.113.1:12345  ◄──  93.184.216.34:80
192.168.1.5:4500  ◄──  (looks up table, rewrites dst)
```

NAT table entry: `(internal IP, internal port)` ↔ `(public IP, public port)`

NAT types: Full cone (most permissive) → Restricted cone → Port-restricted cone → Symmetric (most restrictive). Symmetric NAT breaks peer-to-peer — forces use of TURN relays (relevant for WebRTC).

IPv6

- 128 bits = e.g. `2001:0db8:85a3::8a2e:0370:7334`
- 2^128 = 340 undecillion addresses — effectively unlimited
- No NAT needed; end-to-end addressing restored
- Adoption ~40% of internet traffic (Google data)

4.3 3. TCP vs UDP

TCP (Transmission Control Protocol)

TCP provides a **reliable, ordered, byte-stream** abstraction over unreliable IP.

TCP Header (20 bytes minimum):

```
[Source Port 16b][Dest Port 16b]
[Sequence Number 32b           ]
[Acknowledgment Number 32b     ]
[Offset][Reserved][Flags][Window Size 16b]
[Checksum 16b][Urgent Pointer 16b]
[Options (variable)]
```

Key flags: **SYN** (synchronize), **ACK** (acknowledge), **FIN** (finish), **RST** (reset), **PSH** (push), **URG** (urgent)

Three-Way Handshake (connection establishment):

```
Client                          Server
  |                               |
  |──────── SYN (seq=x) ─────────►|   Client: "I want to connect, my seq starts at x"
  |                               |
  |◄─── SYN-ACK (seq=y,ack=x+1)──|   Server: "OK, my seq starts at y, got your x"
  |                               |
  |──── ACK (seq=x+1,ack=y+1) ──►|   Client: "Got it, we're connected"
  |                               |
  |         [DATA FLOWS]          |
```

Why 3-way and not 2-way? Both sides need to agree on initial sequence numbers. The third ACK confirms the server's SYN was received.

Four-Way Teardown (connection close):

```
Client                          Server
  |                               |
  |──────── FIN (seq=x) ─────────►|   "I'm done sending"
  |◄──────── ACK (ack=x+1) ──────|   "Got it"
  |                               |   [Server may still send data]
  |◄──────── FIN (seq=y) ─────────|   "I'm done too"
  |──────── ACK (ack=y+1) ────────►|   "Got it — connection closed"
  |                               |
  [Client waits 2×MSL (TIME_WAIT)]
```

TIME_WAIT (2×MSL ≈ 60-120s): Ensures delayed packets don't confuse future connections using same port. Can cause ephemeral port exhaustion under high load.

Flow Control (receiver-driven):

- Receiver advertises a **window size** (buffer capacity remaining)
- Sender cannot send more bytes than the window allows
- Prevents fast sender from overwhelming slow receiver
- **Zero window** → sender stops until receiver sends window update

Congestion Control (network-driven):

- Prevents sender from overwhelming the *network* (not just receiver)
- **Slow Start:** Begin with 1 MSS, double cwnd (congestion window) each RTT
- **Congestion Avoidance:** When cwnd > ssthresh, increase by 1 MSS per RTT (linear)
- **Congestion detected by:** packet loss (timeout or 3 duplicate ACKs)
- **TCP Reno:** On 3 dup ACKs → halve cwnd (fast recovery); on timeout → reset to 1
- **TCP CUBIC (Linux default):** cwnd grows as cubic function of time since last loss

```
cwnd
  |    /
  |   /  [Slow Start]
  |  /
  | / ──────────────── ssthresh
  |/         [Congestion Avoidance - linear]
  |
  +──────────────────────────── RTTs
```

UDP (User Datagram Protocol)

UDP provides **unreliable, unordered datagrams** — just source/dest port + payload + checksum.

UDP Header (8 bytes only):

```
[Source Port 16b][Dest Port 16b]
[Length 16b     ][Checksum 16b ]
[Data...                       ]
```

No connection, no sequencing, no retransmission, no congestion control.

TCP vs UDP Comparison Table

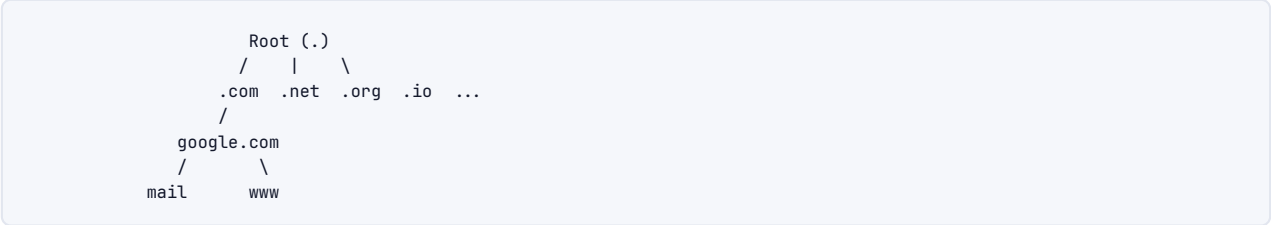
Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Guaranteed delivery + retransmission	No guarantee
Ordering	In-order delivery	May arrive out of order
Overhead	20+ byte header, RTT to establish	8-byte header, no setup
Flow control	Yes (window-based)	No
Congestion control	Yes (slow start, CUBIC)	No
Speed	Slower (reliability cost)	Faster
Use cases	HTTP, SMTP, SSH, FTP	DNS, video streaming, gaming, VoIP
Head-of-line blocking	Yes (one lost packet stalls stream)	No (each datagram independent)

Interview takeaway: Choose UDP when: latency > reliability (live video, games), you're implementing your own reliability (QUIC), or you need multicast/broadcast. Choose TCP for everything else.

4.4 4. DNS (Domain Name System)

DNS is the **phone book of the internet** — maps human-readable names to IP addresses.

DNS Hierarchy



DNS Record Types

Record	Purpose	Example
A	Hostname → IPv4	`google.com → 142.250.80.78`
AAAA	Hostname → IPv6	`google.com → 2607:f8b0:...`
CNAME	Alias → canonical name	`www.example.com → example.com`
MX	Mail server for domain	`example.com → mail.example.com`
NS	Nameserver for domain	`example.com → ns1.google.com`
TXT	Arbitrary text	SPF records, domain verification
PTR	Reverse lookup (IP → hostname)	`78.80.250.142.in-addr.arpa → google.com`
SOA	Start of Authority — zone metadata	Serial number, refresh intervals
SRV	Service location	`_http._tcp.example.com → 80 www.example.com`

DNS Resolution Flow (Full Recursive)

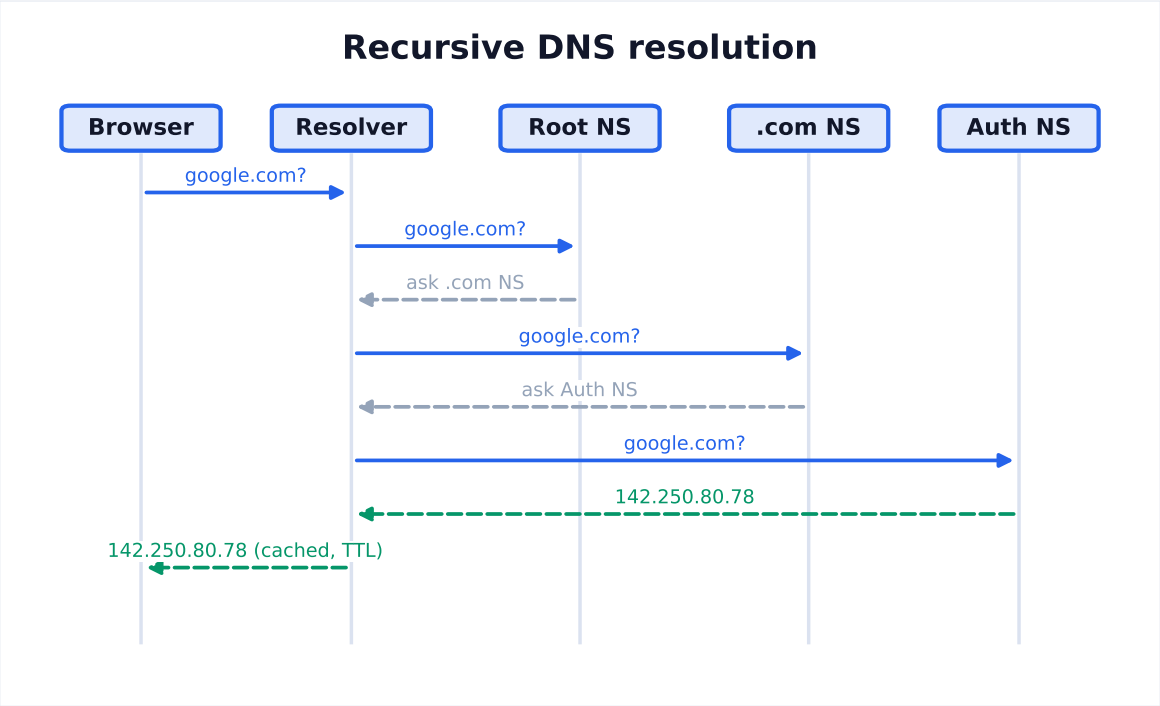


Figure 2. The resolver does the legwork: it walks the hierarchy (root → TLD → authoritative) on the client's behalf, then caches the answer for the TTL.

Caching at every level:

- Browser DNS cache (chrome://net-internals/#dns)
- OS DNS cache / stub resolver
- Recursive resolver (ISP or 8.8.8.8) — largest cache
- Each response has a **TTL** (time-to-live in seconds); lower TTL = faster propagation of changes but more queries

DNS over HTTPS (DoH) / DNS over TLS (DoT): Encrypts DNS queries to prevent eavesdropping and manipulation. Default unencrypted DNS uses UDP port 53.

DNSSEC: Adds cryptographic signatures to DNS records. Prevents DNS spoofing/cache poisoning.

4.5 5. HTTP --- HyperText Transfer Protocol

HTTP Methods

Method	Idempotent	Safe	Purpose
GET	Yes	Yes	Retrieve resource
HEAD	Yes	Yes	GET but headers only
POST	No	No	Create resource / submit data
PUT	Yes	No	Replace entire resource
PATCH	No	No	Partial update
DELETE	Yes	No	Remove resource
OPTIONS	Yes	Yes	Discover allowed methods (CORS)
CONNECT	No	No	Tunnel (used for HTTPS through proxy)

Idempotent: Calling N times = calling 1 time (same result). Safe = read-only, no side effects.

Interview trap: POST is not idempotent (double-submit creates two orders). PUT is idempotent (setting resource to same state twice). DELETE is idempotent (deleting what's gone still = gone).

HTTP Status Codes

```
1xx  Informational     100 Continue, 101 Switching Protocols
2xx  Success           200 OK, 201 Created, 204 No Content
3xx  Redirection       301 Moved Permanently, 302 Found (temp), 304 Not Modified
4xx  Client Error      400 Bad Request, 401 Unauthorized, 403 Forbidden,
                       404 Not Found, 405 Method Not Allowed, 409 Conflict,
                       422 Unprocessable Entity, 429 Too Many Requests
5xx  Server Error      500 Internal Server Error, 502 Bad Gateway,
                       503 Service Unavailable, 504 Gateway Timeout
```

Interview tips:

- 401 = not authenticated (send credentials), 403 = authenticated but not authorized
- 301 = permanent redirect (browsers cache), 302 = temporary (don't cache)
- 502 = load balancer can't reach upstream, 503 = server busy/down, 504 = upstream timeout

Key HTTP Headers

Python

```
Request headers:
  Host: www.example.com
  Authorization: Bearer <token>
  Content-Type: application/json
  Accept: application/json
  Cache-Control: no-cache
  If-None-Match: "abc123"  (conditional GET)
  Cookie: session=abc
  User-Agent: Mozilla/5.0...
  Origin: https://app.example.com  (CORS)
  Connection: keep-alive

Response headers:
  Content-Type: application/json; charset=utf-8
  Cache-Control: max-age=3600, public
  ETag: "abc123"
  Set-Cookie: session=xyz; HttpOnly; Secure; SameSite=Strict
  X-RateLimit-Remaining: 99
  Access-Control-Allow-Origin: https://app.example.com  (CORS)
  Strict-Transport-Security: max-age=31536000  (HSTS)
```

4.6 6. HTTPS and TLS/SSL

Why TLS Exists

Plain HTTP is sent in clear text. Anyone on the network path (ISP, coffee shop WiFi, rogue router) can:

1. Read your data (confidentiality violation)
2. Modify your data (integrity violation)
3. Impersonate the server (authentication violation)

TLS fixes all three.

TLS 1.3 Handshake (Step by Step)

TLS 1.3 (current standard) completes in **1 RTT** (vs TLS 1.2's 2 RTTs).

```
Client                                    Server
  |                                         |
  |─── ClientHello ──────────────────────►|
  |     (TLS version, supported cipher suites, |
  |      key_share: ephemeral DH public key,   |
  |      random nonce)                         |
  |                                         |
  |◄── ServerHello ──────────────────────|
  |     (chosen cipher suite,                  |
  |      key_share: server DH public key)      |
  |                                         |
  |◄── {Certificate} ────────────────────|  [encrypted with derived key]
  |◄── {CertificateVerify} ──────────────|  [server signs handshake transcript]
  |◄── {Finished} ───────────────────────|
  |                                         |
  |     [Client derives session keys from DH]  |
  |     [Client verifies certificate chain]    |
  |                                         |
  |─── {Finished} ──────────────────────►|
  |                                         |
  |======= Encrypted Application Data ========|
```

Key concepts:

1. Asymmetric Cryptography (used for key exchange and auth only):

- Server has a public/private key pair
- Certificate = public key + identity + CA signature
- Used once to authenticate server and establish shared secret

2. Key Exchange — ECDHE (Elliptic Curve Diffie-Hellman Ephemeral):

- Both sides generate ephemeral (temporary) DH key pairs
- Exchange public keys → independently compute same shared secret
- ”Ephemeral” = new key pair per session → **Perfect Forward Secrecy**
- If private key is later compromised, past sessions can't be decrypted

3. Symmetric Encryption (used for all data):

- Derived from DH shared secret using HKDF (key derivation function)
- AES-GCM-256 or ChaCha20-Poly1305 in TLS 1.3
- Much faster than asymmetric crypto

4. Certificate Chain (Root of Trust):

```
Root CA cert (in OS/browser trust store)
  └── Intermediate CA cert (signed by Root CA)
        └── Server cert for example.com (signed by Intermediate CA)
```

Browser walks up chain verifying each signature until it reaches a trusted root.

Certificate Transparency (CT): All publicly-trusted TLS certs must be logged in public append-only logs. Prevents misissuance.

OCSP/CRL: Mechanisms to check if a certificate has been revoked.

TLS 1.2 vs TLS 1.3:

Feature	TLS 1.2	TLS 1.3
Handshake RTTs	2 RTTs	1 RTT
0-RTT resumption	Session tickets (weak)	0-RTT (replay risk)
Cipher suites	Many (including weak ones)	Only strong ones
Forward secrecy	Optional (RSA key exchange!)	Mandatory (ECDHE only)

Feature	TLS 1.2	TLS 1.3
Key exchange	RSA or DH	ECDHE only
Handshake encryption	Partly encrypted	Fully encrypted from ServerHello

4.7 7. REST APIs

REST (Representational State Transfer) is an architectural style, not a protocol, defined by Roy Fielding's 2000 dissertation.

Six REST Constraints

1. **Client-Server:** Separation of concerns — UI and data storage evolve independently
2. **Stateless:** Each request contains all necessary state; server holds no session
3. **Cacheable:** Responses declare themselves cacheable or not
4. **Uniform Interface:** Resources identified by URIs; HATEOAS (optional in practice)
5. **Layered System:** Client can't tell if connected directly to server or via proxy/LB
6. **Code on Demand (optional):** Server can send executable code (JavaScript)

REST API Design Principles

```
Resource: /users
  GET    /users              → list users
  POST   /users              → create user
  GET    /users/{id}         → get user
  PUT    /users/{id}         → replace user
  PATCH  /users/{id}         → update user
  DELETE /users/{id}         → delete user

Nested resources:
  GET    /users/{id}/orders  → orders for user
  POST   /users/{id}/orders  → create order for user

Filtering/pagination:
  GET /products?category=electronics&page=2&limit=20&sort=price_asc

Versioning:
  /api/v1/users   (URI versioning — most common)
  Accept: application/vnd.company.v2+json  (content negotiation)
  X-API-Version: 2  (header versioning)
```

4.8 8. gRPC

gRPC (Google Remote Procedure Call) is a high-performance, open-source RPC framework using **Protocol Buffers** for serialization and **HTTP/2** for transport.

How it Works

```python
Service definition (proto file):
  service UserService {
    rpc GetUser(GetUserRequest) returns (User);
    rpc StreamUsers(Empty) returns (stream User);
  }

Proto compiler generates:
  - Client stub (in any language)
  - Server interface to implement
  - Serialization/deserialization code
```

Protocol Buffers (Protobuf):

- Binary serialization (vs JSON's text)
- Schema-defined, strongly typed
- Typically 3-10x smaller than JSON, 5-10x faster to serialize
- Fields identified by numbers, not names → backward compatible

Four communication patterns:

1. **Unary:** Single request → single response (like HTTP REST)
2. **Server streaming:** Single request → stream of responses
3. **Client streaming:** Stream of requests → single response
4. **Bidirectional streaming:** Both sides stream simultaneously

REST vs gRPC vs GraphQL

Feature	REST	gRPC	GraphQL
Protocol	HTTP/1.1 or 2	HTTP/2	HTTP/1.1 or 2
Format	JSON/XML (text)	Protobuf (binary)	JSON
Typing	No schema (OpenAPI optional)	Strongly typed .proto	Strongly typed schema
Streaming	Limited (SSE)	Full bidirectional	Subscriptions (WebSocket)
Performance	Good	Excellent	Good
Overfetch/underfe	Common	Possible	Solved by design
Browser support	Native	Needs grpc-web proxy	Native
Learning curve	Low	Medium	Medium-High
Best for	Public APIs, simple CRUD	Internal microservices, low latency	Complex queries, mobile clients
Versioning	URI/header	Proto fields (additive)	Schema evolution
Code generation	Optional (OpenAPI)	Required	Optional

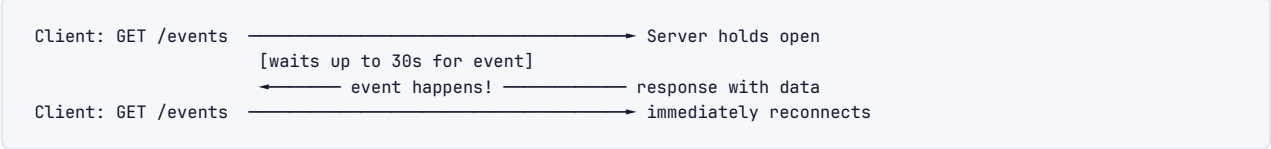
Interview takeaway: Use REST for public-facing APIs (human-readable, universal tooling). Use gRPC for internal microservice-to-microservice calls (performance, type safety, streaming). Use GraphQL when clients have diverse data needs and you want to avoid N+1 REST calls.

4.9 9. WebSockets, SSE, and Long-Polling

The Problem

HTTP is **request-response**: client asks, server answers. For real-time applications (chat, live dashboards, games), you need server-to-client push without repeated polling.

Long-Polling



Pros: Works everywhere (plain HTTP). **Cons:** High latency (event waits for next request), server holds many open connections, head-of-line blocking.

Server-Sent Events (SSE)

```Python
Client: GET /events
Server: Content-Type: text/event-stream

data: {"type":"price_update","value":150.23}\n\n
data: {"type":"price_update","value":150.50}\n\n
```

Pros: Simple, native browser support (EventSource API), HTTP/2 multiplexing, automatic reconnection. **Cons: One-directional** (server → client only), text-only (HTTP/2 adds binary), max 6 connections per domain on HTTP/1.1.

WebSockets

WebSockets upgrade an HTTP connection to a **full-duplex, bidirectional, persistent** TCP connection.

Upgrade handshake:

```
Client → Server:
  GET /chat HTTP/1.1
  Upgrade: websocket
  Connection: Upgrade
  Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
  Sec-WebSocket-Version: 13

Server → Client:
  HTTP/1.1 101 Switching Protocols
  Upgrade: websocket
  Connection: Upgrade
  Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=

[Now raw WebSocket frames flow bidirectionally over TCP]
```

WebSocket Frame:

```
FIN|RSV|Opcode|Mask|Payload Length|Masking Key|Payload
```

Opcodes: 0x1=text, 0x2=binary, 0x8=close, 0x9=ping, 0xA=pong

Comparison: Polling vs SSE vs WebSocket

Feature	Long-Polling	SSE	WebSocket
Direction	Bidirectional (request each way)	Server → Client	Full duplex
Protocol	HTTP	HTTP	WS (TCP upgrade)
Latency	~500ms+	Low	Very low
Overhead	High (repeated headers)	Low	Low (small frames)
Reconnection	Manual	Automatic	Manual
Proxy/firewall	Works everywhere	Works	Sometimes blocked
Load balancer	Easy (stateless)	Easy	Sticky sessions needed
Browser support	Universal	Modern browsers	Modern browsers
Use case	Legacy fallback	Live feeds, notifications	Chat, games, collaboration

Feature	Long-Polling	SSE	WebSocket
Scalability	Harder at scale	Easier	Harder (stateful connections)

Interview takeaway: WebSockets for truly interactive real-time (chat, multiplayer games, collaborative editing). SSE for one-way push (stock tickers, news feeds, notifications). Long-polling for maximum compatibility.

4.10 10. HTTP/1.1 vs HTTP/2 vs HTTP/3

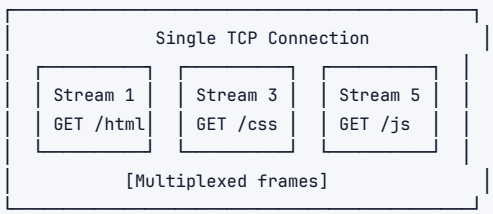
HTTP/1.1 Problems

```
Request 1: ──────────────────────────────────────► wait ◄── Response 1
Request 2: ──────────────────────────────────────► wait ◄── Response 2
                    [sequential — head-of-line blocking]

Workaround: browsers open 6 parallel TCP connections per domain
            → pipelining spec exists but broken in practice
```

HTTP/2 Solutions

HTTP/2 (2015) revolutionizes transport with **binary framing** over a single TCP connection.



HTTP/2 Features:

1. **Multiplexing:** Multiple streams over one TCP connection; no HOL blocking at HTTP level
2. **Binary framing:** Machine-parseable binary vs text (faster, less error-prone)
3. **Header compression (HPACK):** Headers are compressed using a shared table; repeated headers cost ~1 byte
4. **Server Push:** Server proactively sends resources client will need (e.g., push CSS with HTML)
5. **Stream prioritization:** Client can weight streams for priority ordering

HTTP/2 limitation: Still uses TCP → TCP-level head-of-line blocking. One lost packet stalls ALL streams.

HTTP/3 (QUIC)

HTTP/3 replaces TCP with **QUIC** (Quick UDP Internet Connections), built on **UDP**.

```
HTTP/1.1:  [HTTP text]  over  [TCP]   over  [IP]
HTTP/2:    [HTTP/2 binary]  over  [TCP]   over  [IP]
HTTP/3:    [HTTP/3 frames]  over  [QUIC]  over  [UDP]  over  [IP]
```

QUIC bakes in:

- **TLS 1.3** as part of the protocol (0-RTT or 1-RTT total setup)
- **Stream multiplexing** where each stream has independent loss recovery → no TCP HOL blocking

- **Connection migration:** Connection ID persists even when IP/port changes (mobile handoff, WiFi → LTE)
- **Forward Error Correction** (optional): send redundant data to recover from loss without retransmission

QUIC connection setup:

```
HTTP/1.1+TLS 1.2:  TCP(1.5 RTT) + TLS(2 RTT)  = 3.5 RTT before data
HTTP/2+TLS 1.3:    TCP(1 RTT) + TLS(1 RTT)     = 2 RTT before data
HTTP/3+QUIC:       QUIC+TLS(1 RTT)             = 1 RTT before data
                   (0-RTT resumption)           = 0 RTT before data
```

HTTP Comparison Table

Feature	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC (UDP)
TLS	Optional	Optional	Mandatory
Multiplexing	No (one req/conn)	Yes	Yes
HOL blocking	Yes (HTTP)	Yes (TCP)	No
Header compression	No	HPACK	QPACK
Server push	No	Yes (rarely used)	Yes
Connection setup	TCP + TLS separate	TCP + TLS separate	Combined 1-RTT
0-RTT	No	No	Yes (with caveats)
Connection migration	No	No	Yes (QUIC CID)
Adoption	Universal	~60%	~25% growing

4.11 11. Load Balancing

Why Load Balancing

A single server can handle N requests/second. To handle 10N, you need 10 servers + a **load balancer** to distribute traffic.

L4 vs L7 Load Balancing

Dimension	L4 (Transport Layer)	L7 (Application Layer)
OSI Layer	4 (TCP/UDP)	7 (HTTP/gRPC)
What it sees	IP addresses, ports	URLs, headers, cookies, body
Decision basis	IP + port only	URL path, Host header, cookie
Speed	Very fast (minimal processing)	Slower (parse HTTP)
Sticky sessions	By IP (coarse)	By cookie (precise)
SSL termination	Pass-through or offload	Always terminates
Examples	AWS NLB, HAProxy TCP mode	AWS ALB, nginx, Envoy, Traefik
Use case	High throughput, non-HTTP	HTTP routing, canary deploys
Health checks	TCP connect	HTTP endpoint check

L4 flow:

```
Client ──► L4 LB ──► Server A
           (sees IP:port, picks backend, NATs packet)
```

L7 flow:

```
Client ──► L7 LB (terminates TCP+TLS, parses HTTP)
           ├─ /api/users ──► User Service
           ├─ /api/orders ──► Order Service
           └─ /static/* ──► CDN / S3
```

Load Balancing Algorithms

Algorithm	How it works	Best for
Round Robin	Requests distributed in order: A, B, C, A, B, C…	Uniform server capacity
Weighted Round Robin	A gets 2x traffic of B if weight=2	Mixed server capacity
Least Connections	Route to server with fewest active connections	Variable request duration
Least Response Time	Route to fastest responding server	Heterogeneous services
IP Hash	hash(client_IP) % N → consistent server	Basic session affinity
Random	Pick random server	Simple, avoids thundering herd
Consistent Hashing	Ring-based hash, minimal disruption on add/remove	Cache affinity (CDNs)
Resource-based	Route by server CPU/memory utilization	Compute-intensive workloads

Health Checks

Load balancers continuously probe backends:

- **Active:** LB sends HTTP GET /health every N seconds
- **Passive:** LB monitors real traffic, marks backend down on repeated errors

Circuit Breaker pattern: After N failures, stop sending traffic to a backend for M seconds (fast-fail instead of timeout cascades). Libraries: Hystrix, Resilience4j.

Topology Options

```
Single LB (SPOF):
  Clients ──► [LB] ──► [Server A]
                   ──► [Server B]

HA LB pair (active-passive):
  Clients ──► [LB-Primary] ──► [Server A]
          ──► [LB-Secondary]   [Server B]
              (standby; takes over via Virtual IP on failure)

Multi-tier:
  Clients ──► [Global LB / DNS] ──► [Region A LB] ──► [App Servers]
                                ──► [Region B LB] ──► [App Servers]
```

4.12 12. CORS (Cross-Origin Resource Sharing)

Same-Origin Policy (SOP): Browsers prevent JavaScript on `https://app.example.com` from making requests to `https://api.other.com` — different origin.

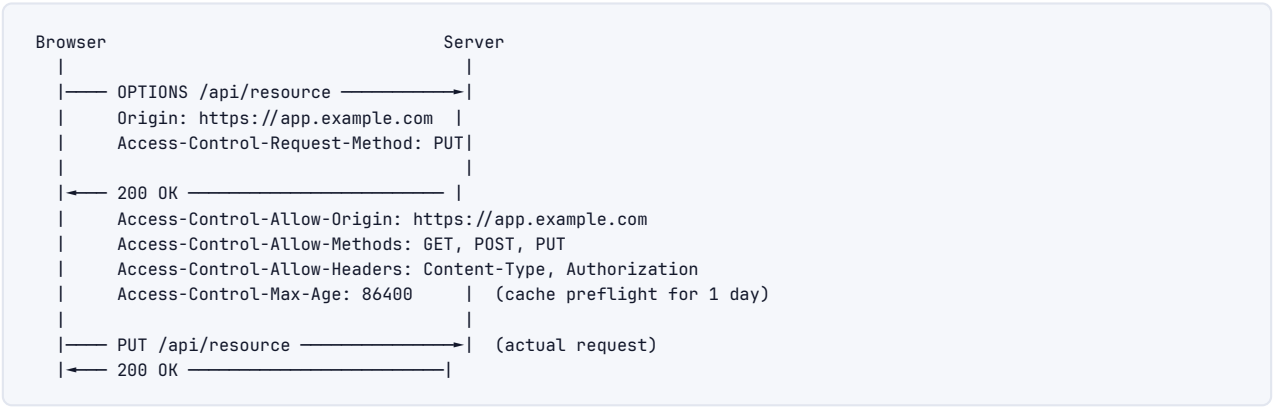
Origin = scheme + host + port: `https://app.example.com:443` vs `https://api.example.com:443` = different origins (different subdomain).

CORS solves this by letting the server declare which origins are permitted.

Simple vs Preflight Requests

Simple request (GET/POST with basic headers): Browser sends request with `Origin` header, checks `Access-Control-Allow-Origin` in response.

Preflight request (PUT/DELETE/custom headers, content-type: application/json):



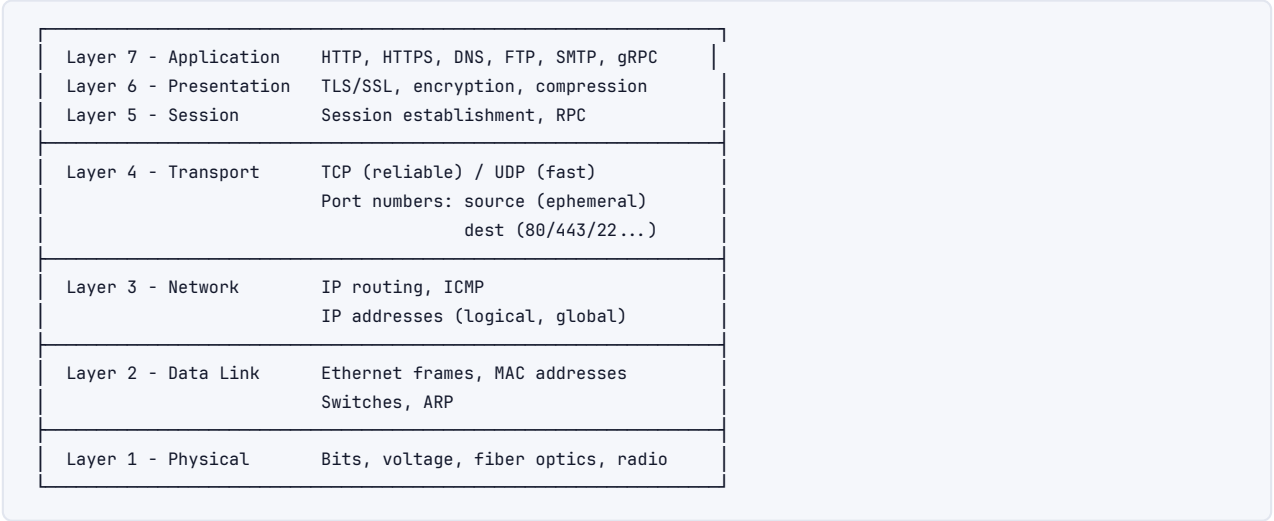
Key CORS headers:

- `Access-Control-Allow-Origin: *` or specific origin
- `Access-Control-Allow-Methods: GET, POST, PUT, DELETE`
- `Access-Control-Allow-Headers: Content-Type, Authorization`
- `Access-Control-Allow-Credentials: true` (can't combine with `*` origin)
- `Access-Control-Max-Age: 3600` (cache preflight duration)

Interview trap: CORS is a **browser security mechanism** — it doesn't protect your API server. Server-to-server requests bypass CORS entirely. CORS headers tell the browser what's allowed; the browser enforces it.

5 Architecture / Diagrams

5.1 OSI Stack Diagram



5.2 TCP 3-Way Handshake

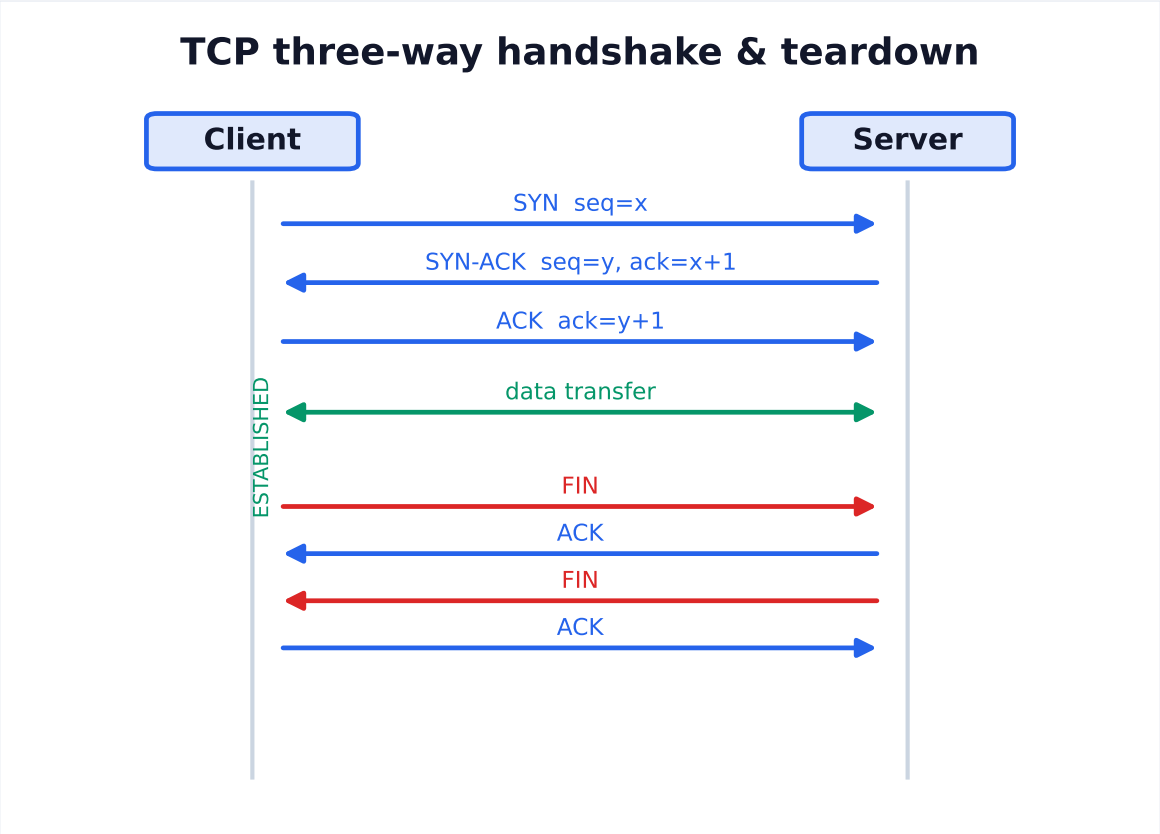


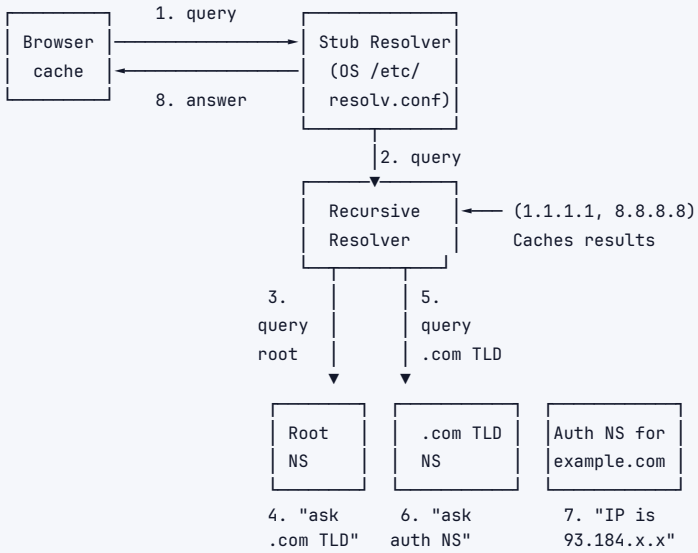
Figure 3. TCP opens with SYN / SYN-ACK / ACK to agree on sequence numbers, then tears down with a four-way FIN/ACK exchange. This reliability is what UDP trades away for speed.

```
Client              Network              Server
  │                                         │
  │  SYN(seq=1000)                          │
  │─────────────────────────────────────── ►│
  │                                         │  Server: ISN=2000
  │               SYN-ACK(seq=2000,ack=1001)│
  │◄─────────────────────────────────────── │
  │                                         │
  │  ACK(seq=1001,ack=2001)                 │
  │─────────────────────────────────────►   │
  │                                         │
  │          [Connection Established]       │
  │◄═══════════════════════════════════════►│
```

5.3 TLS 1.3 Handshake

```
Client                                  Server
  │                                       │
  │──── ClientHello ──────────────────►   │
  │     [key_share, random, cipher_suites] │
  │                                       │
  │◄──── ServerHello ─────────────────    │
  │     [key_share, chosen_cipher]        │
  │                                       │
  │◄═══ {Certificate + Verify + Finished} =  [encrypted]
  │                                       │
  │     [Client: compute shared secret]   │
  │     [Client: verify cert chain]       │
  │                                       │
```

5.4 DNS Resolution Flow

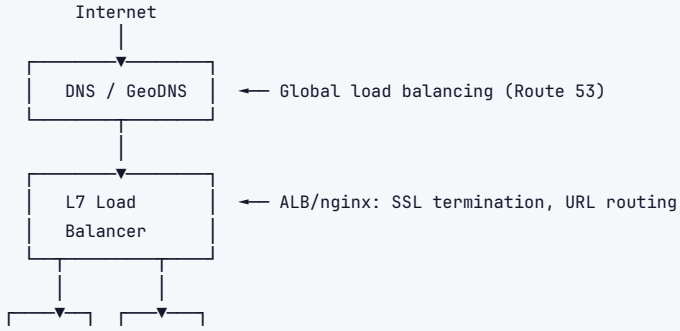


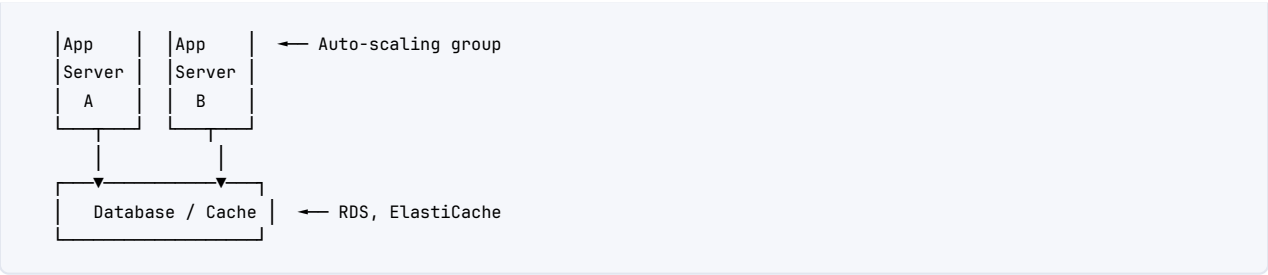
5.5 HTTP/2 Multiplexing

```
HTTP/1.1 — 6 parallel TCP connections needed:
  TCP Conn 1: [Request 1]──────────[Response 1]
  TCP Conn 2: [Request 2]──────────[Response 2]
  TCP Conn 3: [Request 3]──────────[Response 3]
  (...)

HTTP/2 — 1 TCP connection, multiple streams:
  ┌──────────────────────────────────────┐
  │          Single TCP Connection       │
  │  Frame: [Hdr S1][Data S3][Hdr S2][D S1] │
  │          interleaved binary frames   │
  └──────────────────────────────────────┘
  Stream 1: GET /index.html → HTML
  Stream 2: GET /style.css  → CSS
  Stream 3: GET /app.js     → JS
  (all in parallel, in-order per stream)
```

5.6 Load Balancer Topology





6 Real-World Examples

Google Search: Your browser makes an HTTP/3 GET request. Before that, DNS resolves google.com (likely cached). TLS 1.3 negotiated in 1 RTT. Response arrives in ~100ms because Google's servers are globally distributed via anycast.

Netflix video streaming: Client starts with DASH or HLS (HTTP-based adaptive bitrate). L7 load balancer routes to nearest Open Connect CDN node. If a TCP packet is lost mid-stream → HTTP/2 head-of-line blocking can stall playback → why Netflix pushed for QUIC/HTTP/3.

Slack real-time messaging: WebSocket connections maintained per-client to Slack's servers. Messages go through Kafka, then pushed to WebSocket handlers. Connection affinity needed — sticky sessions or L7 LB that understands WebSocket upgrades.

AWS API Gateway: L7 load balancer + API management. Routes `GET /users` to Lambda A, `POST /orders` to Lambda B. Handles SSL termination, rate limiting, auth.

CloudFlare: Acts as L7 reverse proxy + DDoS protection. Your traffic hits CloudFlare edge → TLS terminated there → re-encrypted to your origin. CloudFlare's anycast network means clients connect to nearest PoP.

7 Real-Life Analogies

One postal network — every concept is a parcel, a courier, or a depot in the same delivery system.

Concept	Analogy
OSI Layers	A parcel moving through the postal network: you write the contents (application), seal and label the box (presentation/session), write the full street address (network), hand it to the courier service (transport), the courier van carries it link by link (data link), and tarmac roads physically connect the depots (physical)
TCP vs UDP	TCP = registered mail with a signature required on delivery — if no one signs, the courier re-attempts until it gets through. UDP = dropping a postcard in the public letterbox — fast, no tracking number, some never arrive, and you never find out
TCP 3-way handshake	The courier calls ahead before dispatching a big shipment: ”Parcel coming your way — ready to receive?” → ”Yes, loading bay is clear” → ”Great, truck is leaving now.” Only after that three-beat exchange does the lorry set off
TCP congestion control	A courier fleet entering a congested motorway: start with one van (slow start), add a van each trip as the road clears (linear growth), then a pile-up ahead forces the dispatcher to cut the convoy in half and rebuild carefully from there
DNS	The national address directory: you know the company's trading name (”GlobalWidgets Ltd”) but need its actual street address before you can send a parcel — the directory looks it up and hands back the postcode
DNS TTL	How long a courier is allowed to use a cached address before re-checking the directory. Short TTL = re-check every day (safe if the company moves often). Long TTL = re-check yearly (risky — parcels may land at a vacated office)

Concept	Analogy
TLS certificate	A notarized identity card stapled to the parcel: a government-recognized authority has stamped "this parcel truly originates from GlobalWidgets Ltd." Both sender and recipient trust the stamp because they both recognize the authority that issued it
Diffie-Hellman key exchange	Two couriers agree on a secret seal color without ever meeting: each mixes their private dye into a shared base color, swaps the result, and mixes again — they end up with an identical hue that no eavesdropper at the sorting depot can reproduce
Perfect Forward Secrecy	The depot shreds the wax seal mold after every shipment. Even if a thief later steals the master key, all past parcels are already sealed with unique, destroyed molds and cannot be reopened
Load balancer	The dispatch desk at the sorting depot: every inbound parcel lands on the desk, and the dispatcher hands it to whichever courier van is free right now, keeping the workload spread evenly across the fleet
Round-robin LB	The dispatcher assigns parcels in strict rotation — van A, van B, van C, van A again — so every courier takes the same share of the queue regardless of how heavy each parcel is
Least connections LB	The dispatcher checks which van has the fewest parcels still in transit and loads the next one onto that van, so no courier is buried while another idles
HTTP/2 multiplexing	A single courier van carrying parcels for many recipients simultaneously, sorted into labeled compartments — versus the old rule where the van had to deliver one parcel completely before picking up the next
QUIC/HTTP/3	The postal authority builds its own private courier road with built-in security checkpoints and dedicated lane markings — bypassing the congested public motorway (TCP) entirely, so vans never queue behind someone else's blocked lorry
WebSockets	Opening a permanent, two-way courier hatch between two depots: parcels flow in both directions any time, without the sender having to knock and wait for a fresh door-opening each time
CORS	The receiving depot's gate policy: when a parcel arrives from a third-party courier (a different origin), the gatekeeper checks the approved-senders list posted at the gate. If that courier company is not listed, the parcel is turned away — not destroyed, just refused entry
NAT	The company mailroom: 200 staff share a single public street address on the building's front door. Every outgoing parcel is relabeled with the building's address; the mailroom log records which desk it came from so the reply can be routed back to exactly the right person
CDN	Regional depots stocked with copies of the most popular goods: instead of every parcel shipping from the central warehouse hundreds of miles away, customers in each region collect from the nearest local depot — same goods, fraction of the journey

8 Memory Tricks / Mnemonics

8.1 OSI Layers (Top to Bottom)

"All People Seem To Need Data Processing"

- **A**pplication (7)
- **P**resentation (6)
- **S**ession (5)
- **T**ransport (4)
- **N**etwork (3)
- **D**ata Link (2)

- **P**hysical (1)

  Bottom to Top: **"Please Do Not Throw Sausage Pizza Away"**

8.2 TCP Flags

"Unskilled Attackers Pester Real Security Folk"

- **U**RG, **A**CK, **P**SH, **R**ST, **S**YN, **F**IN

8.3 HTTP Status Code Families

"Information, Success, Redirect, Client-fault, Server-fault"

- 1xx = Info
- 2xx = Success
- 3xx = Redirect
- 4xx = Client error (your fault)
- 5xx = Server error (their fault)

8.4 TCP Handshake

SYN → **SYN-ACK** → **ACK** = **"See ya!"** → **"See ya, and you!"** → **"You!"**

8.5 REST HTTP Methods → CRUD

- **C**reate = POST
- **R**ead = GET
- **U**pdate = PUT/PATCH
- **D**elete = DELETE

8.6 TLS 1.3 in one line

"Hello → **Hello+Keys** → **Cert+Verify+Done** → **Done** → **Data"**

8.7 DNS hierarchy

Root → **TLD** → **Auth** → **Answer** = **"Root's Totally Authoritative Answers"**

8.8 Load Balancing levels

"Layer 4 is Fewer, Layer 7 is Flexible"

- L4 = Fast, few options (IP+port)
- L7 = Feature-rich (headers, URLs, cookies)

8.9 CIDR quick math

- /24 = 256 addresses = one office
- /16 = 65,536 addresses = one building
- /8 = 16M addresses = one ISP
- Each bit you remove from prefix → doubles the addresses

9 Common Interview Questions

9.1 Q1: "What happens when you type `https://www.google.com` in your browser and press Enter?"

This is the most comprehensive networking question. A complete answer demonstrates end-to-end mastery.

Step 1 — URL Parsing Browser parses: scheme=`https`, host=`www.google.com`, port=`443` (implied), path=/

Step 2 — DNS Resolution

```
Browser checks: in-memory cache → OS cache → /etc/hosts
If not cached:
  → stub resolver → recursive resolver (e.g., 8.8.8.8)
  → recursive resolver asks: root NS → .com TLD NS → google.com auth NS
  → gets A record: 142.250.80.78, TTL=300
  → all layers cache it
```

Step 3 — TCP Connection

```
Browser → TCP SYN to 142.250.80.78:443
Server  → TCP SYN-ACK
Browser → TCP ACK
(connection established in ~50ms for nearby servers)
```

Step 4 — TLS Handshake (TLS 1.3)

```
Browser → ClientHello (cipher suites, key_share)
Server  → ServerHello (chosen cipher, key_share) + Certificate + CertificateVerify + Finished
Browser → verifies cert chain (root CA in OS trust store)
        → derives session keys from ECDHE
        → sends Finished
(1 RTT total for first connection; 0-RTT possible on resumption)
```

Step 5 — HTTP Request

Python

```
GET / HTTP/2
Host: www.google.com
Accept: text/html,application/xhtml+xml
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Cookie: [existing cookies]
```

Step 6 — Server Processing

- Hits L7 load balancer (terminates TLS)
- Routes to one of many Google frontend servers
- Server checks cache, assembles response
- May query backend microservices

Step 7 — HTTP Response

Python

```
HTTP/2 200 OK
Content-Type: text/html; charset=UTF-8
Content-Encoding: br (Brotli compressed)
Cache-Control: private, max-age=0
[HTML body]
```

Step 8 — Browser Rendering

- HTML parsed → DOM tree
- Browser discovers CSS/JS/font/image URLs → additional HTTP/2 streams (multiplexed)
- CSSOM built → Render tree → Layout → Paint
- Page displayed

Step 9 — Connection Stays Alive

- HTTP/2 connection persists (keep-alive)
- Subsequent requests reuse connection without new TCP+TLS handshake
- WebSocket upgrade if page uses real-time features

Follow-up Q: "What if the DNS resolves to multiple IPs?" → Browser tries the first IP. If connection fails, tries next (Happy Eyeballs for IPv4/IPv6). Load balancer or anycast routes to nearest data center.

Follow-up Q: "What is HSTS and when does it kick in?" → After first HTTPS visit, server sends `Strict-Transport-Security` header. Browser caches this and automatically upgrades future HTTP requests to HTTPS for that domain (for max-age duration).

9.2 Q2: "Explain TCP flow control vs congestion control"

Flow control solves: receiver overwhelmed by fast sender.

- Receiver advertises window size (buffer space remaining)
- Sender must not send more unacknowledged bytes than the window
- If receiver fills up → window = 0 → sender pauses
- **Mechanism:** Receiver-advertised window in TCP header

Congestion control solves: network overwhelmed (routers dropping packets).

- Sender maintains cwnd (congestion window)
- Slow start: cwnd doubles each RTT until ssthresh
- Congestion avoidance: cwnd grows by 1 MSS per RTT
- On loss: cwnd cut in half (Reno) or CUBIC curve
- **Mechanism:** Inferred from packet loss or ECN marks

Key distinction: Flow control = receiver tells you to slow down. Congestion control = sender figures out network is congested.

9.3 Q3: "Why does TLS use both asymmetric and symmetric cryptography?"

Asymmetric crypto (RSA, ECDH): mathematically expensive but lets two parties establish a shared secret without ever transmitting it. Cannot be used for bulk data — too slow (1000x slower than symmetric).

Symmetric crypto (AES-GCM): extremely fast (hardware acceleration, ~10GB/s), but requires both parties to already share a key.

Solution: Use asymmetric crypto *once* to establish a shared secret, then use symmetric crypto for all data. This is called a **hybrid cryptosystem** and is how all practical secure communication works.

9.4 Q4: "REST vs gRPC --- when do you choose each?"

Choose REST when:

- Public-facing API that any client (browser, mobile, curl) must consume
- Team is small, protocol simplicity matters
- Resource-based semantics fit naturally

- Caching of responses is important (HTTP cache headers)
- API will be consumed by external partners with diverse toolchains

Choose gRPC when:

- Internal microservice-to-microservice communication
- Low latency and high throughput required
- Bidirectional streaming needed
- Polyglot services (proto generates clients in any language)
- Strong schema enforcement reduces bugs across service boundaries

Follow-up: "What's the cost of gRPC?"

- No native browser support (needs grpc-web proxy or Envoy)
- Harder to debug (binary, not human-readable)
- Tighter coupling (proto changes require coordination)
- Less mature ecosystem for HTTP-level tooling (caching, analytics)

9.5 Q5: "What is HTTP/2 head-of-line blocking and how does HTTP/3 fix it?"

HTTP/1.1 HOL blocking: Only one request per connection at a time. Solution: open 6 parallel connections (wasteful).

HTTP/2 HOL at TCP level: HTTP/2 solved HTTP-level HOL with multiplexing, but all streams share one TCP connection. A single lost TCP packet → all streams stall while TCP retransmits that packet. Streams aren't independent at the transport layer.

HTTP/3 fix: Replace TCP with QUIC (UDP-based). QUIC implements streams at the protocol level. A lost packet only stalls the stream that contained it — other streams continue flowing. True end-to-end elimination of HOL blocking.

9.6 Q6: "Explain the difference between L4 and L7 load balancing. When do you use each?"

L4 Load Balancer:

- Operates at TCP/UDP layer
- Sees only IP addresses and port numbers
- Routes based on IP+port; can't inspect content
- Typically faster (less processing)
- SSL pass-through or terminate and re-establish
- Examples: AWS NLB, LVS, HAProxy TCP mode
- **Use when:** Need maximum throughput, non-HTTP protocols (database, MQTT, game servers)

L7 Load Balancer:

- Operates at HTTP/HTTPS layer
- Can inspect URLs, headers, cookies, body
- Route `/api` to one backend, `/static` to another
- SSL termination (decrypts before routing)
- Can do auth, rate limiting, header manipulation
- Examples: AWS ALB, nginx, Envoy, Traefik
- **Use when:** Need content-based routing, canary deployments, A/B testing, microservices

9.7 Q7: "What is CORS and why does it exist? How is it different from authentication?"

CORS exists to relax Same-Origin Policy. SOP prevents JS on evil.com from making credentialed requests to bank.com. Without SOP, malicious site could steal your banking session via AJAX.

CORS is NOT authentication. It only controls browser-enforced cross-origin access. A server-side API call from curl or Postman completely bypasses CORS — the browser enforces it, not the server.

CORS headers tell the browser "JavaScript from origin X is allowed to read my responses." Without the header, the browser blocks the *response* (the request still reaches the server!).

Common mistake: Thinking CORS provides security. It doesn't — it enables access in a controlled way. Your API still needs proper auth regardless.

10 Senior-Level Discussion Points

1. QUIC and UDP reliability paradox UDP is unreliable, yet QUIC on UDP provides reliability. QUIC implements its own: sequence numbers, ACKs, retransmission, congestion control — but all at the application layer with stream-level independence. This shows that reliability is software, not a hardware necessity.

2. TCP connection establishment cost in microservices If service A calls service B on every request with a new TCP+TLS connection: 1 RTT (TCP) + 1 RTT (TLS 1.3) = 2 RTTs overhead before first byte. At 50ms RTT, that's 100ms pure overhead. Solution: **connection pooling** (maintain a pool of warm TCP+TLS connections). gRPC's HTTP/2 transport multiplexes many calls over few connections — this is a key architectural advantage at scale.

3. Consistent hashing in load balancing Standard round-robin: adding one server → all sessions remapped (cache stampede). Consistent hashing: adding one server → only K/N sessions move (where K = sessions on that shard, N = total servers). Critical for cache-sensitive workloads.

4. Zero-RTT and replay attacks HTTP/3 QUIC supports 0-RTT resumption — client sends data in the first packet without waiting. Risk: attacker replays that packet (e.g., GET is safe, POST /transfer is not). 0-RTT data must be idempotent. TLS 1.3 provides replay protection mechanisms but 0-RTT is inherently weaker.

5. DNS as a global load balancer GeoDNS (Weighted Round Robin, latency-based) routes users to nearest data center. DNS TTL becomes critical: low TTL = fast failover but more DNS load; high TTL = stale entries during incidents. AWS Route 53 supports sub-second health check-driven DNS failover.

6. TLS certificate rotation in microservices mTLS (mutual TLS): both client and server present certificates. Used in service meshes (Istio, Envoy). Short-lived certificates (24h) via SPIFFE/SPIRE replace long-lived certs, reducing breach windows. Certificate rotation without downtime requires careful orchestration.

7. Epoll and high connection counts Classic Unix `select()` is O(N) per call — breaks above 1000 connections. `epoll` (Linux) and `kqueue` (BSD) are O(1) for event notification. Node.js, Nginx, Go all use epoll to handle 100K+ concurrent connections on a single server. WebSocket servers hit connection limits without this.

8. TCP time-wait and port exhaustion Each closed TCP connection stays in TIME_WAIT for 2×MSL (~60s). High-throughput systems (>1000 connections/sec) exhaust the 64K ephemeral port range. Solutions: `SO_REUSEADDR`, `TCP_TIMESTAMPS + tw_reuse`, multiple IPs, or connection pooling.

11 Typical Mistakes Candidates Make

1. **Confusing 401 and 403:** 401 = not authenticated (send credentials). 403 = authenticated but not authorized. Many candidates mix these up.
2. **Saying PUT is same as POST:** PUT replaces the entire resource at a URI (idempotent). POST creates a new resource (not idempotent). PATCH partially updates.
3. **Saying TLS is slow:** TLS 1.3 with hardware AES-NI is negligible overhead. The TLS handshake cost matters for short connections — solution is connection reuse, not avoiding TLS.
4. **Thinking HTTP/2 eliminates all head-of-line blocking:** HTTP/2 eliminates HTTP-level HOL but TCP-level HOL remains. HTTP/3/QUIC solves TCP HOL.
5. **Confusing flow control and congestion control:** Flow control = receiver's buffer limit. Congestion control = network capacity limit. Different problems, different mechanisms.
6. **Saying CORS protects APIs:** CORS is browser-enforced. Server-to-server requests ignore CORS entirely. Real API protection = authentication + authorization.
7. **Ignoring DNS caching in system design:** Candidates design failover systems but forget DNS TTL means it can take minutes for changes to propagate. **FAANG expects you to mention TTL**.
8. **Not mentioning connection pooling:** Naively suggesting "each microservice call opens a new TCP connection" shows inexperience at scale.
9. **Forgetting about TLS certificate chain verification:** Just saying "TLS encrypts data" misses the authentication step — verifying the server is who it claims to be.
10. **Conflating WebSocket and HTTP/2 server push:** Different mechanisms. WebSocket = bidirectional, full-duplex. HTTP/2 push = server sends resources proactively before client asks, still unidirectional.
11. **Subnetting math errors:** Know that /24 = 256 IPs (254 usable), /25 = 128 IPs, /26 = 64 IPs. Each bit added to prefix halves the address space.
12. **Saying "add more servers" without discussing the load balancer:** More servers require a load balancer. Candidates should discuss health checks, algorithms, and session affinity.

12 How This Connects to Other Topics

12.1 System Design

- **Horizontal scaling** requires load balancers — L4 vs L7 choice matters
- **Caching** at CDN (L7), reverse proxy (HTTP Cache-Control, ETags), application level
- **Database connections** = TCP connections with connection pooling (PgBouncer)
- **API design** = REST vs gRPC vs GraphQL — system design question embedded
- **Real-time features** = WebSockets or SSE — stateful connections affect scaling

12.2 Distributed Systems

- **Partition tolerance:** Network partitions are the P in CAP theorem
- **Timeouts and retries:** TCP's congestion control doesn't help at application level
- **Service discovery:** DNS-based (Kubernetes services) or sidecar (Consul, Envoy)
- **Circuit breakers:** Complement to load balancing when backends fail

12.3 Security

- **TLS everywhere** (zero trust): mTLS between microservices
- **Certificate pinning:** Mobile apps pin expected cert to prevent MITM

- **HTTP security headers:** HSTS, CSP, X-Frame-Options, Referrer-Policy
- **OAuth/JWT over HTTPS:** Token-based auth relies on TLS for security
- **DDoS mitigation:** Anycast + rate limiting at L3/L4 (SYN flood protection)

12.4 Cloud (AWS/GCP/Azure)

- **AWS NLB** = L4 load balancer (target by IP, handles TCP/UDP)
- **AWS ALB** = L7 load balancer (HTTP routing, WebSocket, gRPC)
- **Route 53** = DNS + global load balancing (geolocation, latency, health)
- **VPC** = virtual network with private IP spaces, NAT gateways, security groups (stateful L4 firewall)
- **API Gateway** = L7 proxy + rate limiting + auth + SSL termination
- **CloudFront** = CDN + L7 features (Lambda@Edge)

12.5 Performance Engineering

- **Latency budget:** DNS lookup + TCP handshake + TLS + request + response — each step counts
- **Connection reuse:** keep-alive, connection pools, HTTP/2 multiplexing
- **Compression:** gzip/Brotli for text, Protobuf for structured data
- **TTFB (Time to First Byte):** Metric for server responsiveness, affected by routing/LB

13 FAANG Interview Tips

1. Think out loud, always. Networking questions are diagnostic. Show your mental model even when uncertain.

2. Start at the right layer. When asked "why is X slow?" → systematically go through: DNS? TCP? TLS? Network bandwidth? Server processing? Database?

3. "What happens when you type a URL" is the universal opening. Master it to the level in this guide. It demonstrates DNS, TCP, TLS, HTTP, and rendering knowledge.

4. Volunteer trade-offs. Never say "use X" without saying "but the cost is Y." REST → simple but no streaming. gRPC → fast but no browser support. HTTP/3 → fast but UDP blocked in some networks.

5. Know the numbers:

- RTT for same-AZ: <1ms, same-region: 1-5ms, cross-region: 20-100ms
- DNS TTL common: 300s (5 min), some use 60s, some 3600s
- TLS 1.3: 1 RTT. TLS 1.2: 2 RTT
- TCP handshake: 1 RTT
- HTTP/1.1 max connections per domain: 6 (browsers)

6. Connect networking to system design. "I'd use WebSockets for the chat feature, but that requires sticky sessions on the load balancer, or a shared pub-sub backend like Redis to broadcast messages across server instances."

7. Idempotency comes up constantly. Know why it matters for retries: safe to retry idempotent operations; retrying non-idempotent ones (POST) causes duplicate records. Include idempotency keys in API design discussions.

8. For Amazon: Know VPC, subnets, security groups, NACLs. They'll relate networking to AWS primitives.

9. For Google: Be comfortable with QUIC, SPDY, and the trade-offs of building on UDP. Google invented QUIC — interviewers know it deeply.

10. Security angle: Always mention TLS. "And of course all communication would be over TLS" should be reflexive.

14 Revision Cheat Sheet

14.1 10-Minute Summary

OSI: 7 layers, Physical → Data Link → Network → Transport → Session → Presentation → Application. Mnemonic: "Please Do Not Throw Sausage Pizza Away."

TCP: Reliable, ordered, connection-oriented. 3-way handshake (SYN/SYN-ACK/ACK). 4-way close. Flow control (receiver window). Congestion control (slow start, CUBIC).

UDP: Unreliable, unordered, connectionless. 8-byte header. Use for: DNS, video, games, QUIC.

DNS: Hierarchical distributed DB. Types: A, AAAA, CNAME, MX, NS, TXT, PTR. Resolution: stub → recursive → root → TLD → auth → answer.

TLS 1.3: 1 RTT. ClientHello (key_share) → ServerHello+Cert+Verify+Finished (encrypted) → Client Finished → Data. ECDHE for key exchange → Perfect Forward Secrecy. Cert chain verified against root CA in trust store.

HTTP Methods: GET (safe, idempotent), POST (create, not idempotent), PUT (replace, idempotent), PATCH (partial update), DELETE (idempotent). Status: 2xx=success, 3xx=redirect, 4xx=client error (401=unauthed, 403=forbidden), 5xx=server error.

REST: Stateless, resource-based, HTTP verbs. Good for public APIs.

gRPC: Protocol Buffers + HTTP/2. Binary, strongly typed, streaming, fast. Good for internal microservices.

WebSockets: Upgrade HTTP connection to full-duplex TCP. Use for real-time bidirectional (chat, games). SSE for server-push only. Long-polling for maximum compatibility.

HTTP/1.1 vs 2 vs 3: 1.1=text, one req/conn. 2=binary+multiplexed+HPACK, TCP HOL. 3=QUIC(UDP)+no HOL+0-RTT+connection migration.

Load Balancing: L4=IP+port, fast. L7=HTTP-aware, content routing. Algorithms: Round Robin, Least Connections, Consistent Hashing. Health checks required.

CORS: Browser enforces same-origin policy. CORS headers let server allow cross-origin browser requests. Preflight (OPTIONS) for non-simple requests. NOT a security mechanism for APIs.

NAT: Private IPs (10.x, 172.16-31.x, 192.168.x) mapped to public IPs via translation table on gateway router.

CIDR: /N means N bits are network prefix. Usable hosts = 2^(32-N) - 2.

14.2 Key Numbers to Memorize

Item	Value
IPv4 address bits	32 bits
IPv6 address bits	128 bits
TCP header min size	20 bytes
UDP header size	8 bytes
Ethernet MTU	1500 bytes
HTTP port	80
HTTPS port	443
DNS port	53 (UDP and TCP)
SSH port	22
TLS 1.3 handshake	1 RTT
TLS 1.2 handshake	2 RTTs
TIME_WAIT duration	~60 seconds (2×MSL)

Item	Value
Browser TCP conns per domain	6 (HTTP/1.1)
/24 subnet usable hosts	254
/16 subnet addresses	65,536

14.3 Master Comparison Table

Protocol	Layer	Transport	Format	Streaming	Browser	Use Case
REST	7	TCP	Text (JSON)	No (SSE only)	Yes	Public APIs
gRPC	7	TCP (HTTP/2)	Binary (Protobuf)	Bidirectional	Via proxy	Internal RPC
GraphQL	7	TCP	Text (JSON)	Subscriptions	Yes	Flexible queries
WebSocke	7	TCP (upgrade)	Text/Binary	Full duplex	Yes	Real-time
SSE	7	TCP (HTTP)	Text	Server→Client	Yes	Live feeds
DNS	7	UDP (mostly)	Binary	No	N/A	Name resolution
HTTP/1.1	7	TCP	Text	No	Yes	Web
HTTP/2	7	TCP	Binary	Yes (streams)	Yes	Web (modern)
HTTP/3	7	QUIC/UDP	Binary	Yes (streams)	Yes (75%)	Web (fast)
TCP	4	—	Binary	Byte stream	—	Reliable transport
UDP	4	—	Binary	Datagrams	—	Fast transport
IP	3	—	Binary	Packets	—	Routing

14.4 Most Important Concepts (FAANG Top-10)

1. **Full URL walkthrough** (DNS → TCP → TLS → HTTP → Response) — can appear in any form
2. **TCP 3-way handshake** — always asked when discussing connections
3. **TLS 1.3 handshake** — security-focused companies always ask
4. **TCP flow control vs congestion control** — fundamental transport knowledge
5. **HTTP methods + idempotency** — embedded in API design questions
6. **REST vs gRPC trade-offs** — comes up in every microservices system design
7. **WebSocket vs SSE vs polling** — comes up in every real-time feature design
8. **HTTP/1.1 vs 2 vs 3 differences** — "how would you improve performance?"
9. **L4 vs L7 load balancing** — mandatory in any scaling discussion
10. **CORS mechanism** — front-end + API integration, security questions